4 TTC methods comparison

Tuesday, February 2026 11:58 am

Comparison table

Notation used throughout:

- $I_p(t)$: intensity of pixel p at time/frame t
- $p = 1, \ldots, N$: pixels in the ROI
- $\langle \cdot \rangle_p$: average over pixels
- $\bar{I}_p = \langle I_p(t) \rangle_t$: time-average of pixel p
- $S(t) = \sum_p I_p(t)$: summed ROI intensity at time t

Method name	What it averages over	Pre-processing	Math definition	Normalization	Sensitive to
G-TTC	Pixels in ROI	None (raw intensities)	$G(t_1,t_2) = \frac{\langle I(t_1)I(t_2)\rangle - \langle I(t_1)\rangle\langle I(t_2)\rangle}{\sigma(t_1)\sigma(t_2)}$	Per-frame variance normalization	Fluctuations around the mean, contrast changes
Corr-TTC	Pixels in ROI	None (raw intensities)	$\mathrm{Corr}(t_1,t_2) = \frac{\langle I(t_1)I(t_2)\rangle}{\langle I(t_1)\rangle\langle I(t_2)\rangle}$	Mean-intensity normalization only	Intensity correlations including slow drifts
Twotime.py TTC	Pixels in ROI	Per-pixel temporal smoothing	$C(t_1,t_2) = \frac{1}{N}\sum_p \frac{I_p(t_1)}{\bar{I}_p}\frac{I_p(t_2)}{\bar{I}_p}\frac{N}{S(t_1)S(t_2)}$	Per-pixel mean + per-frame sum	Relative pixel-wise fluctuations, suppresses static structure
CGPT TTC	Pixels in ROI	Per-pixel temporal smoothing	Same as twotime.py	Same as twotime.py	Same as twotime.py

Legend

- 🟢 ✓ = captures / supported
- ❌ = does not capture
- ⚠️ = captures **indirectly or conditionally** (important nuance)

Feature	G	Corr	twotime.py	CGPT
Slow drift	❌	❌❌	🟢 ✓	🟢 ✓
Oscillatory dynamics	🟢 ✓	🟢 ✓	🟢 ✓	🟢 ✓
Pixel heterogeneity	🟢 ✓	🟢 ✓	❌	❌
Grain-boundary selectivity	🟢 ✓	🟢 ✓	⚠️	⚠️
Direct comparison to saved HDF	❌	❌	🟢 ✓	🟢 ✓

ChatGPT's twotime-like method

1. What object is being built first: I(t,p)

From the raw detector frames you construct an **intensity matrix**

I(t,p)

where:

- t = frame index (time)
- p = pixel index inside the selected ROI (mask)

This comes from:
I, frame_idxs = extract_roi_intensity_matrix(...)

So at this point:

- Shape: I.shape == (T, P)
- T = number of selected frames
- P = number of pixels in the ROI

Twotime.py APS 8-ID-E method

Inputs

- A stack of raw detector frames for a run
  D(t, y, x)
- A ROI definition (mask) selecting a set of pixels p = 1..P

From this you form the ROI intensity matrix:

I(t,p) \quad \text{shape } (T, P)

where:

- t is frame index (time)
- p is pixel index within the ROI

Step 1: per-pixel smoothing / normalization

This is **exactly** the data structure that twotime.py operates on internally.

2. Per-pixel normalization (“smooth data” step)

Next you apply:

I_smooth = _smooth_like_twotime(I)

Mathematically:

For each pixel p,

$$\bar{I}_p = \frac{1}{T}\sum_{t=1}^{T} I(t,p)$$

Then each pixel’s time trace is normalized by its own mean:

$$I_{\text{smooth}}(t,p) = \frac{I(t,p)}{\bar{I}_p}$$

Key consequences:

- Removes static pixel-to-pixel intensity differences
- Preserves **temporal correlations**
- Makes all pixels contribute equally to the TTC

This step is **crucial** and often overlooked. You got it right.

3. Core two-time correlation (twotime math)

This is the heart of the TTC, computed by:

C_raw_ut = _compute_ttc_like_twotime(I_smooth)

Your helper exactly mirrors twotime.py::calc_normal_twotime.

3.1 Pixel-averaged cross product

First compute the pixel-averaged intensity product:

$$\langle I(t_1)I(t_2)\rangle_p = \frac{1}{P}\sum_{p=1}^{P} I_{\text{smooth}}(t_1,p)\, I_{\text{smooth}}(t_2,p)$$

In matrix form this is:

cross = (I @ I.T) / P

3.2 Frame-wise normalization (important)

Define the **total ROI intensity per frame**:

$$S(t) = \sum_{p=1}^{P} I_{\text{smooth}}(t,p)$$

Then define a normalization factor:

$$n(t) = \frac{1}{S(t)}$$

Compute, for each pixel p, its time-mean:

\bar I(p) = \frac{1}{T}\sum_{t=1}^{T} I(t,p)

Guard against zeros / negatives:

\bar I(p) \leftarrow 1 \quad \text{if } \bar I(p) \le 0

Normalize each pixel’s time series:

I_{\text{smooth}}(t,p) = \frac{I(t,p)}{\bar I(p)}

This step is optional in twotime.py (depends on which path/flags are used), but when enabled it happens **before** computing the TTC.

Step 2: frame-wise normalization factor

Compute the sum over pixels for each frame:

S(t) = \sum_{p=1}^{P} I_{\text{smooth}}(t,p)

Guard:

S(t) \leftarrow 1 \quad \text{if } S(t) \le 0

Define the normalization factor:

n(t) = \frac{1}{S(t)}

So n(t) is a length-T vector.

Step 3: pixel-averaged cross-product matrix

Compute the time–time dot product (average over pixels):

M(t_1, t_2) = \sum_{p=1}^{P} I_{\text{smooth}}(t_1,p)\,I_{\text{smooth}}(t_2,p)

In matrix form:

M = I_{\text{smooth}} \, I_{\text{smooth}}^\top

This yields a T \times T matrix.

Step 4: apply normalization and pixel-count scaling

Let P be the number of pixels in the ROI.

twotime.py forms:

C_2(t_1,t_2) = M(t_1,t_2)\; n(t_1)\; n(t_2)\; P

Equivalently:

C_2 = \left(I_{\text{smooth}} I_{\text{smooth}}^\top\right) \odot \left(n\,n^\top\right) \cdot P

where \odot is elementwise multiplication.

Step 5: keep only the upper triangle

twotime.py stores only:

C_2(t_1,t_2) \quad \text{for } t_2 \ge t_1

i.e.:

C_2 \leftarrow \mathrm{triu}(C_2)

(Your _symmetrize_upper_triangle() reconstructs the full symmetric TTC later for plotting.)

Summary in your implementation terms

This enforces intensity normalization **per frame**, not per pixel.

3.3 Final TTC definition (twotime convention)

The raw two-time correlation is:

$$C(t_1, t_2) = P \langle I(t_1) I(t_2) \rangle_p \, n(t_1) \, n(t_2)$$

Which expands to:

$$C(t_1, t_2) = \frac{\sum_p I_{\text{smooth}}(t_1, p) \, I_{\text{smooth}}(t_2, p)}{\left(\sum_p I_{\text{smooth}}(t_1, p)\right) \left(\sum_p I_{\text{smooth}}(t_2, p)\right)}$$

This is **exactly** what twotime computes.

In the code:

matmul_prod = ts @ ts.T
c2 = matmul_prod * norm_factor[:, None] * norm_factor[None, :] * npix

4. Upper-triangle storage and symmetrization

twotime.py only computes:

t_2 \ge t_1

Which enforces:

C(t_1,t_2) = C(t_2,t_1)

This is correct and necessary for visualization and comparison.

5. What this TTC actually measures (important interpretation)

This TTC is **not**:

\langle I(t_1) I(t_2) \rangle

and it is **not** a standard Pearson correlation.

It is a **normalized pixel-ensemble intensity overlap**, meaning:

- Sensitive to **reproducibility of the speckle pattern**
- Insensitive to static intensity offsets
- Sensitive to slow, collective rearrangements
- Exactly what Bragg-XPCS needs

6. Why your comparison to processed TTC is meaningful

Because:

- Both raw and processed TTCs:
  - use the **same pixel set**
  - use the **same normalization**
  - use the **same frame indexing**
- Any difference you see in RAW − PROCESSED must come from:
  - preprocessing differences
  - masking differences

Summary of your implementation steps

For one ROI:

1. Build I(t,p) from raw frames using the ROI pixel map
2. (Optional) normalize each pixel column by its own time mean
3. Compute frame sums S(t) and n(t)=1/S(t)
4. Compute M = I I^\top
5. Compute C_2 = M \cdot n(t_1)n(t_2)\cdot P
6. Store only triu(C2)

That's the full algorithm you coded in:

- _smooth_like_twotime() → Step 1
- _compute_ttc_like_twotime() → Steps 2–5